



Gestión de Datos

Bases de Datos NoSQL

MongoDB



Características:

- BD de documentos.
- Esquema Flexible.
- Distribuida y resiliente:
 - Distribuida: escalado horizontal (*sharding).
 - Resiliencia: a través de la replicación.

MongoDB Atlas es la **plataforma de base de datos como servicio (DBaaS)** oficial de MongoDB Inc. que permite **usar MongoDB en la nube** sin tener que instalar, configurar ni mantener servidores manualmente.

MongoDB Atlas

ORGANIZATION: UAX PROJECT: Project 0

Clusters

Find a database deployment...

Cluster0 [Connect](#) [View Monitoring](#) [Browse Collections](#) [...](#)

+ Create

Tiene un plan gratuito con:

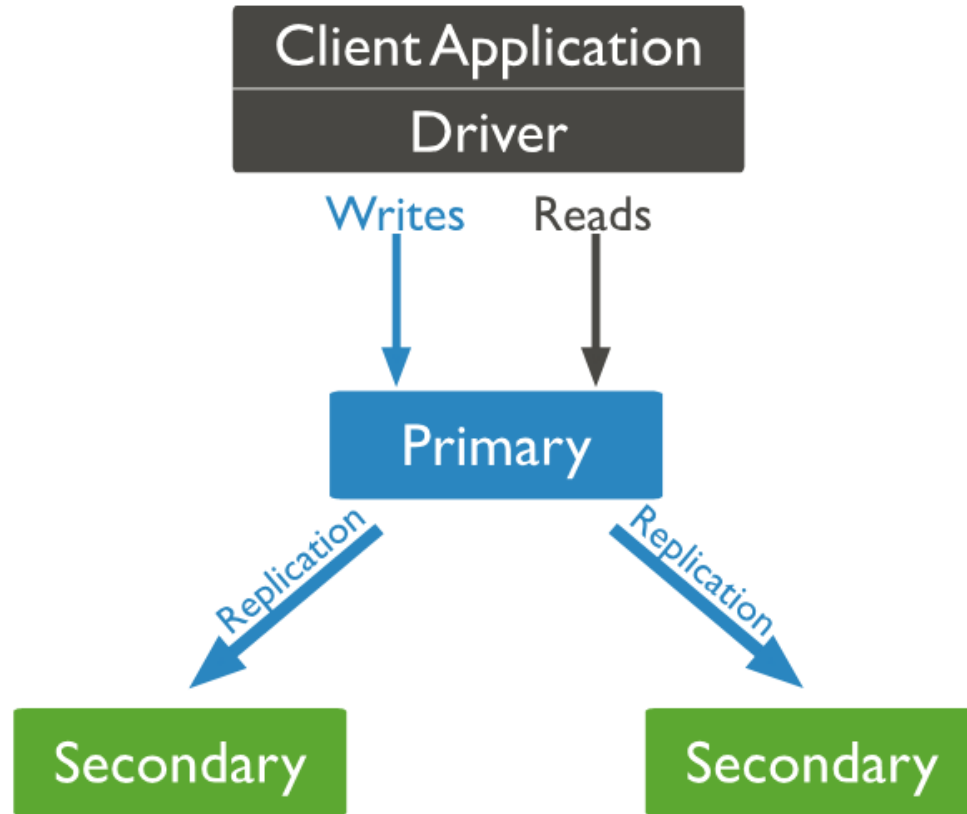
- 512 MB de almacenamiento.
- 1 clúster compartido.
- 3 nodos de réplica.
- Ideal para aprendizaje o pequeños proyectos.

Visualize Your Data

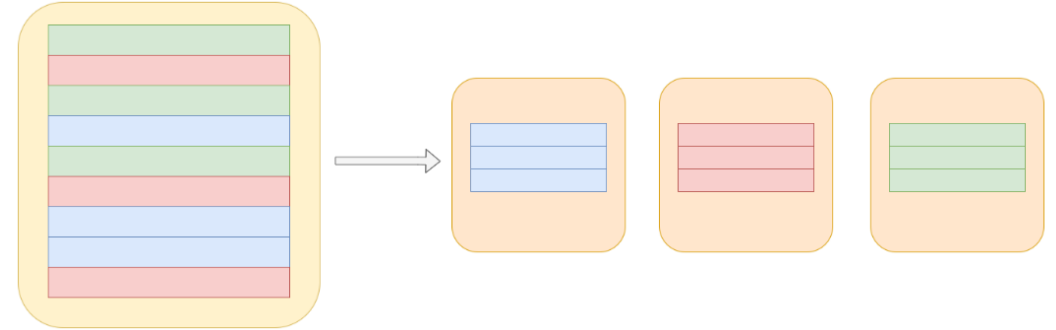
Build dashboards and charts, and embed them in your apps with MongoDB Charts.

[Explore Charts](#) [Dismiss](#)

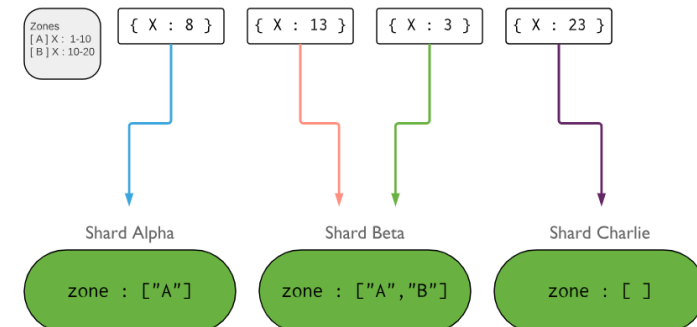
VERSION	REGION	TYPE	BACKUPS	LINKED APP SERVICES	ATLAS SQL	ATLAS SEARCH
8.0.20	AWS / Paris (eu-west-3)	Replica Set - 3 nodes	Inactive	None Linked	Connect	Create Index



Resiliencia: a través de la replicación



Sharding (Fragmentación, Partitioning)
 Distribuye los datos entre los fragmentos (shards) de un cluster.
 Réplicas de shards en sharded clusters.



Dentro del clúster podemos crear diferentes BBDD, que van a estar formadas por 1->n colecciones.

Data Explorer

{ } My Queries

🏗 Data Modeling

CLUSTERS (1)

- Cluster0
 - admin
 - deportes
 - local
 - sample_mflix**
 - comments
 - embedded_movies

sample_mflix +

Cluster0 > sample_mflix

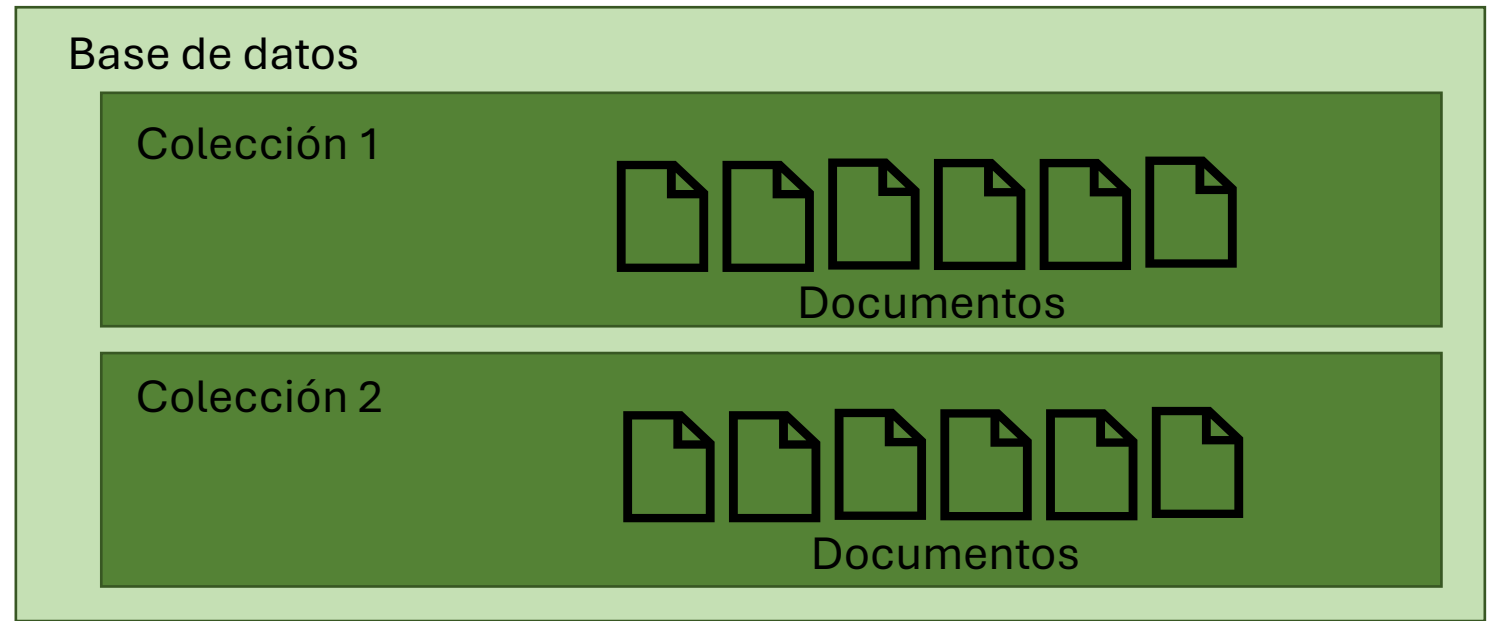
[View monitoring](#)
[Visualize your data](#)
[+ Create collection](#)
[Refresh](#)

Collection name ↕	Properties ↕	Storage size ↕	Data size ↕	Documents	Avg. Document size ↕	Indexes	Total Index size ↕
comments	-	8.22 MB	11.69 MB	41K	284.00 B	1	1.74 MB
embedded_movies	-	82.16 MB	75.39 MB	3.5K	21.64 kB	1	217.09 kB
movies	-	24.26 MB	34.12 MB	21K	1.60 kB	2	18.19 MB
sessions	-	36.86 kB	540.00 B	1	540.00 B	2	57.34 kB
theaters	-	299.01 kB	349.83 kB	1.6K	223.00 B	2	135.17 kB
users	-	69.63 kB	29.57 kB	185	159.00 B	2	65.54 kB

Mod. Relacional	Mongo
Database	Database
Table, View	Collection
Row	Document
Column	Field
Index	Index

La información se almacena en documentos en forma de pares clave:valor.
Esquema flexible:

- Los campos pueden contener datos anidados: objetos o arrays.
- Los campos pueden variar entre documentos de una misma colección.



```
{
  "_id": 1,
  "name": { "first" : "John", "last" : "Backus" },
  "contribs": [ "Fortran", "ALGOL", "Backus-Naur Form", "FP" ],
  "awards": [
    {
      "award": "W.W. McDowell Award",
      "year": 1967,
      "by": "IEEE Computer Society"
    }, {
      "award": "Draper Prize",
      "year": 1993,
      "by": "National Academy of Engineering"
    }
  ]
}
```

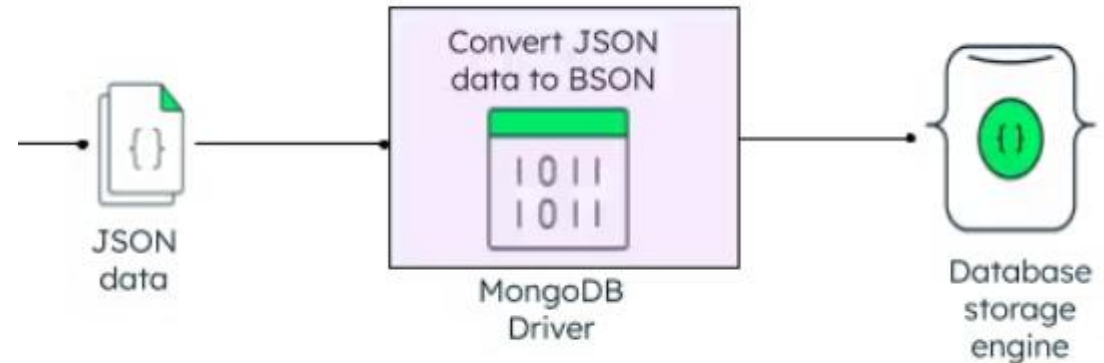
JSON (JavaScript
Object Notation):
legible para
humanos

Mod. Relacional	Mongo
Database	Database
Table, View	Collection
Row	Document
Column	Field
Index	Index

La información se almacena en documentos en forma de pares clave:valor.

Esquema flexible:

- Los campos pueden contener datos anidados: objetos o arrays.
- Los campos pueden variar entre documentos de una misma colección.



```

{"hello": "world"} →
\x16\x00\x00\x00      // total document size
\x02                   // 0x02 = type String
hello\x00              // field name
\x06\x00\x00\x00world\x00 // field value
\x00                   // 0x00 = type E00 ('end of object')

{"BSON": ["awesome", 5.05, 1986]} →
\x31\x00\x00\x00
\x04BSON\x00
\x26\x00\x00\x00
\x02\x30\x00\x08\x00\x00\x00awesome\x00
\x01\x31\x00\x33\x33\x33\x33\x33\x33\x14\x40
\x10\x32\x00\xc2\x07\x00\x00
\x00
\x00
  
```

BSON (Binary JSON): representación binaria de JSON. Facilita la búsqueda transversal de datos.

JSON vs BSON

	JSON	BSON
Encoding	UTF-8 string	Binary
Data Support	String, boolean, number, array, object, null	String, boolean, number (integer, float, long, decimal128...), array, null, date, BinData
Readability	Human and machine	Machine only

```
{
  _id: ObjectId('68e4d6291af77cf4f00a7c24'),
  nombre: 'Pepe',
  apellidos: 'Montes López',
  direccion: {
    calle: 'Gran Vía',
    cp: '28001'
  }
}
{
  _id: ObjectId('68e77560b0fef87db615c254'),
  nombre: 'Pedro'
}
{
  _id: ObjectId('68e4d7b3c2082a77eb48d913'),
  nombre: 'Sara',
  apellidos: 'Sanz Torres',
  visitas: 5,
  profesiones: [
    'profesora',
    'programadora'
  ],
  hobbies: [
    'leer',
    'pasear'
  ]
}
```

Esquema flexible

⇒ Manejo natural de campos nulos o ausentes

⇒ Ideal para:

- Aplicaciones con datos altamente dinámicos
- Modelos de negocios que evolucionan rápido.
- Datos semiestructurados o heterogéneos.

Sin esquema predefinido (Schema-less):

MongoDB **no requiere que definas un esquema de antemano** como en las bases de datos relacionales. Podemos agregar documentos con distintos campos y estructuras dentro de una misma colección.

Compass



{ } My Queries

🏗 Data Modeling

MongoDB – Seguridad y Usuarios

Network & Database Access

	Database & Network...
	DATABASE ACCESS ▲
	Database Users
	Custom Roles
	NETWORK ACCESS ▲
	IP Access List
	Peering
	Private Endpoint
	Advanced

MongoDB – Crear & Borrar BD

Compass (UI)

Create Database ×

Database Name

prueba

Collection Name

prueba

☐ **Time-Series**

Time-series collections efficiently store sequences of measurements over a period of time. [Learn More](#) 

> Additional preferences (e.g. Custom collation, Clustered collections)

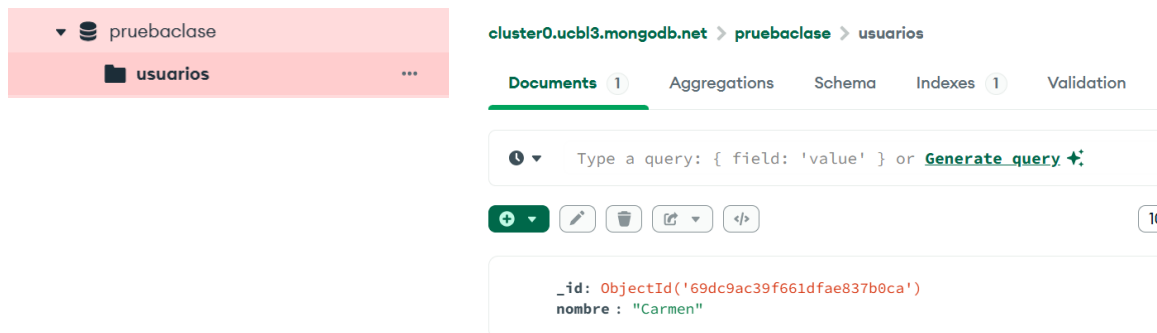
Cancel

Create Database

Es necesario crear una colección para crear una BD.

MongoDB – Crear & Borrar BD

```
> use test
< switched to db test
> use pruebaclase
< switched to db pruebaclase
> db.usuarios.insertOne({nombre:"Carmen"})
< {
  acknowledged: true,
  insertedId: ObjectId('69dc9ac39f661dfae837b0ca')
}
Atlas atlas-vjr5r2-shard-0 [primary] pruebaclase> |
```



MONGOSH (shell)

- Seleccionar una BD (no importa si aun no existe): esto no crea la BD físicamente, únicamente cambia el contexto.
- Inserta un documento en una colección: en este momento, MongoDB crea la BD pruebaclase con una colección usuarios e inserta el documento. Actualizar en Compass para visualizar.

MongoDB – Crear & Borrar BD

```
> db.getCollectionNames()
< [ 'usuarios' ]
> db.createCollection("collection2")
< { ok: 1 }
> db.getCollectionNames()
< [ 'usuarios', 'collection2' ]
```

MONGOSH (shell)

- Ver y crear colecciones en una BD.
- Para ver la información de las colecciones usar `getCollectionInfos()`.

MongoDB – Crear & Borrar BD

Compass (UI)

 **Drop Database?** x

Are you sure you want to drop database "prueba"?

Type "prueba" to confirm your action

Cancel Drop Database

MONGOSH (shell)

```
> db.dropDatabase("pruebaclase")  
< { ok: 1, dropped: 'pruebaclase' }
```


MongoDB – Documentos

Inserción de documentos:

- Se pueden importar y exportar en formatos .csv y .json desde la interfaz de usuario
- Se pueden insertar con consultas `insertOne()` o `insertMany()`. Datos en formato diccionario, en el caso de `insertMany` array (lista) de diccionarios.

MongoDB – Documentos

```
import pymongo
import sys
import os
import dotenv
import pandas as pd
```

```
"""Conexión a MongoDB"""

#cargar las variables de entorno
dotenv.load_dotenv()
strconnectDB = os.getenv("strconnectDB")

#Conexión a BD
try:
    client = pymongo.MongoClient(strconnectDB)
except pymongo.errors.ConfigurationError:
    print("Error de conexión. ....")
    sys.exit(1)
```

Se crea un cliente que se conecta a MongoDB usando la cadena de conexión proporcionada desde Atlas o Compass.

Usamos variables de entorno para ocultar los datos de la cadena de conexión.

MongoDB – Documentos

```
""" Subir datos desde .csv """

#Leer csv
df = pd.read_csv("data/cpmunicipios.csv") #la
url y archivos a probar

#Seleccionar / Crear bd y colección
db = client["munimadrid"]
collection_csv = db["datafromcsv"]

#Subir documentos
docs = df.to_dict(orient="records")
collection_csv.insert_many(docs)

print("Archivo csv subido a MongoDB")
```

Ver en Compass

MongoDB – Documentos

```
""" Subir datos desde .json """  
  
import json  
  
#Leer json  
docs = None  
try:  
    with open("data/cpmunicipios.json", 'r', encoding='utf-8') as archivo:  
        docs = json.load(archivo)  
except FileNotFoundError:  
    print("archivo no encontrado")  
  
#Seleccionar / Crear bd y colección  
db = client["munimadrid"]  
collection_json = db["datafromjson"]  
  
#Subir documentos  
if docs:  
    collection_json.insert_many(docs)  
  
    print("Archivo json subido a MongoDB")
```

Ver en Compass

MongoDB – Documentos

¿Cómo muevo los datos desde SQL a MongoDB?

```
from sqlalchemy import create_engine

#cargar las variables de entorno
dotenv.load_dotenv()
strconnectPSQLDB = os.getenv("strconnectPSQLDB")

#Conectar a PostgreSQL
try:
    # Configurar la conexión
    engine_saleshealth = create_engine(strconnectPSQLDB)

except Exception as e:
    print(f"Error: {e}")
```

Conecto a PostgreSQL con
psycopg2 o sqlalchemy.

MongoDB – Documentos

¿Cómo muevo los datos desde SQL a MongoDB?

```
#Leer una tabla completa
df_tablecustomer =
pd.read_sql_table('customer',engine_saleshealth)
print(df_tablecustomer)

#Seleccionar / Crear bd y colección
db = client["saleshealth"]
col_customer = db["customer"]

#Subir documentos
docs = df_tablecustomer.to_dict(orient="records")
col_customer.insert_many(docs)

print("Documentos subidos a MongoDB")

#Buscar en mongoDB y mostrar algunos resultados
results = list(col_customer.find().limit(5))

print(results)
```

Con pandas y sqlalchemy
leo una tabla, en este
ejemplo customer.

Convertir a diccionario e
insertar en MongoDB.

MongoDB – Documentos

¿Cómo muevo los datos desde SQL a MongoDB?

Comparación de tamaños

PSQL

```
SELECT pg_size_pretty(pg_total_relation_size('customer'));
```

	pg_size_pretty text
1	1232 kB

MongoDB

Collection name ⓘ	Properties ⓘ	Storage size ⓘ	Data size ⓘ	Documents ⓘ	Avg. document size ⓘ	Indexes ⓘ	Total index size ⓘ
customer	-	4.10 kB	1.13 MB	5.8K	196.00 B	1	4.10 kB

```
# Crear colecciones con estructuras de datos anidados
# Producto: generar datos con anidamiento en categoría y marca

col_product = db["product"]

#Leer las tablas
df_tableproduct = pd.read_sql_table('product',engine_saleshealth)
df_tablebrand = pd.read_sql_table('brand',engine_saleshealth)
df_tablecategory = pd.read_sql_table('category',engine_saleshealth)

dict_product = df_tableproduct.to_dict(orient="records")
dict_brand = df_tablebrand.to_dict(orient="records")
dict_category = df_tablecategory.to_dict(orient="records")
```

- Se crea una nueva colección
- Se accede con pandas y sqlalchemy a las tablas
- Se convierten en diccionarios
- Se reemplazan con los datos correspondientes en productos, para ellos se usan los diccionarios de marcas y categorías. En su lugar, podrían lanzarse consultas SQL e incluir los resultados en el diccionario del correspondiente documento.
- Se comprueba y se sube a MongoDB.

```
for product in dict_product:

    catval = product["category"]
    newcatdata = {}
    for cat in dict_category:
        if cat["name"] == catval:
            newcatdata["category_id"] = cat["category_id"]
            newcatdata["category"] = cat["name"]
            exit
    product["category"] = newcatdata

    brandval = product["manufacturer"]
    newbranddata = {}
    for brand in dict_brand:
        if brand["name"] == brandval:
            newbranddata["brand_id"] = brand["brand_id"]
            newbranddata["brand"] = brand["name"]
            newbranddata["country"] = brand["country"]
            exit
    product["manufacturer"] = newbranddata

print(dict_product)

#Subir documentos
col_product.insert_many(dict_product)

print("Documentos subidos a MongoDB")

#Buscar en mongoDB y mostrar algunos resultados
results = list(col_product.find().limit(5))

print(results)
```



```
_id: ObjectId('69dce093bb9cd9f6fad8ab54')
product_id: 1
name: "Tensiómetro digital"
▼ category: Object
  category_id: 1
  category: "Diagnóstico"
▼ manufacturer: Object
  brand_id: 1
  brand: "Omron"
  country: "Japón"
price: 59.99
created_at: 2026-04-04T11:23:48.591+00:00
```

```
_id: ObjectId('69dce093bb9cd9f6fad8ab55')
product_id: 2
name: "Oxímetro de pulso"
▼ category: Object
  category_id: 1
  category: "Diagnóstico"
▼ manufacturer: Object
  brand_id: 2
  brand: "Beurer"
  country: "Alemania"
price: 29.99
created_at: 2026-04-04T11:23:48.591+00:00
```

Resultado visualizado en Compass

MongoDB – Documentos

¿Cómo muevo los datos desde SQL a MongoDB?

Con herramientas de migración.

TOOLS

MongoDB Relational Migrator Download

MongoDB Relational Migrator simplifies and streamlines data migration from legacy relational databases to MongoDB, reducing the effort and risk involved in migration initiatives.

It supports migrations from popular SQL databases, including Oracle, SQL Server, MySQL, PostgreSQL, Sybase, and IBM DB2, to any self-managed MongoDB deployment or MongoDB Atlas.

<https://www.mongodb.com/try/download/relational-migrator>

Relational Migrator

1.15.3

New Project

Start from scratch

Import project



Connect to live database

Connect to a running database to retrieve the schema.

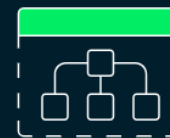
 **Connect database**



Load schema from a file

Upload a SQL file containing Data Definition Language statements.

 **Import .SQL file**



Use a sample schema

Explore Relational Migrator with a sample relational schema.

+ Create sample

Relational Migrator

1.15.3

1 Connect to database

2 Select tables

3 Define initial schema

4 Name project

Connect to the database

To start a new migration job, you will need to reconnect to the live source database by selecting a saved connection, entering your source relational database details, or by manually entering your connection string in the JDBC URI field. [Learn more](#)

Select method

☐ Use a saved connection

☒ Add a new connection

Database type

PostgreSQL

JDBC URI

☒ Enter URI manually

jdbc:postgresql://localhost:5432/saleshealth

Username

uaxuser

Password

.....

☐ Save password ⓘ

Connection name

localhost-PostgreSQL

Environment tag

None

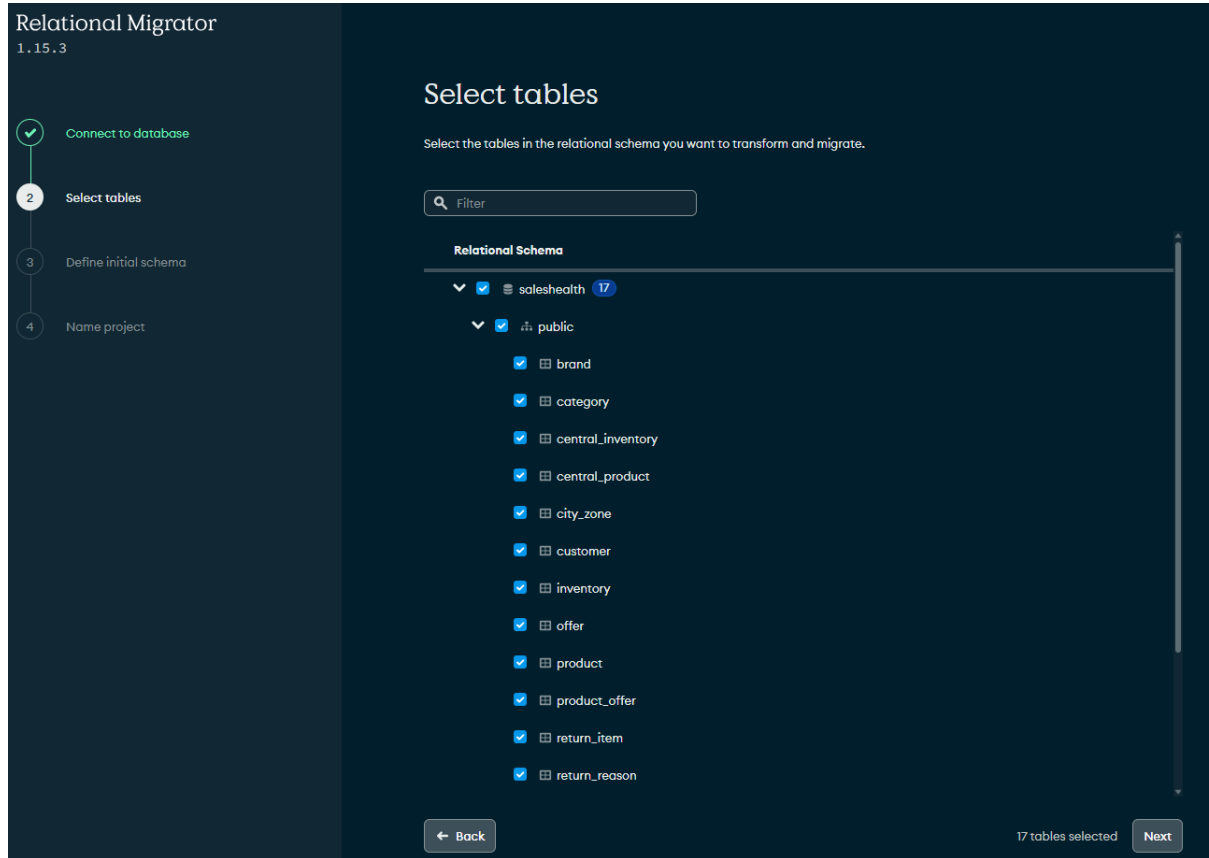
✓ PostgreSQL database connection successful.

← Back

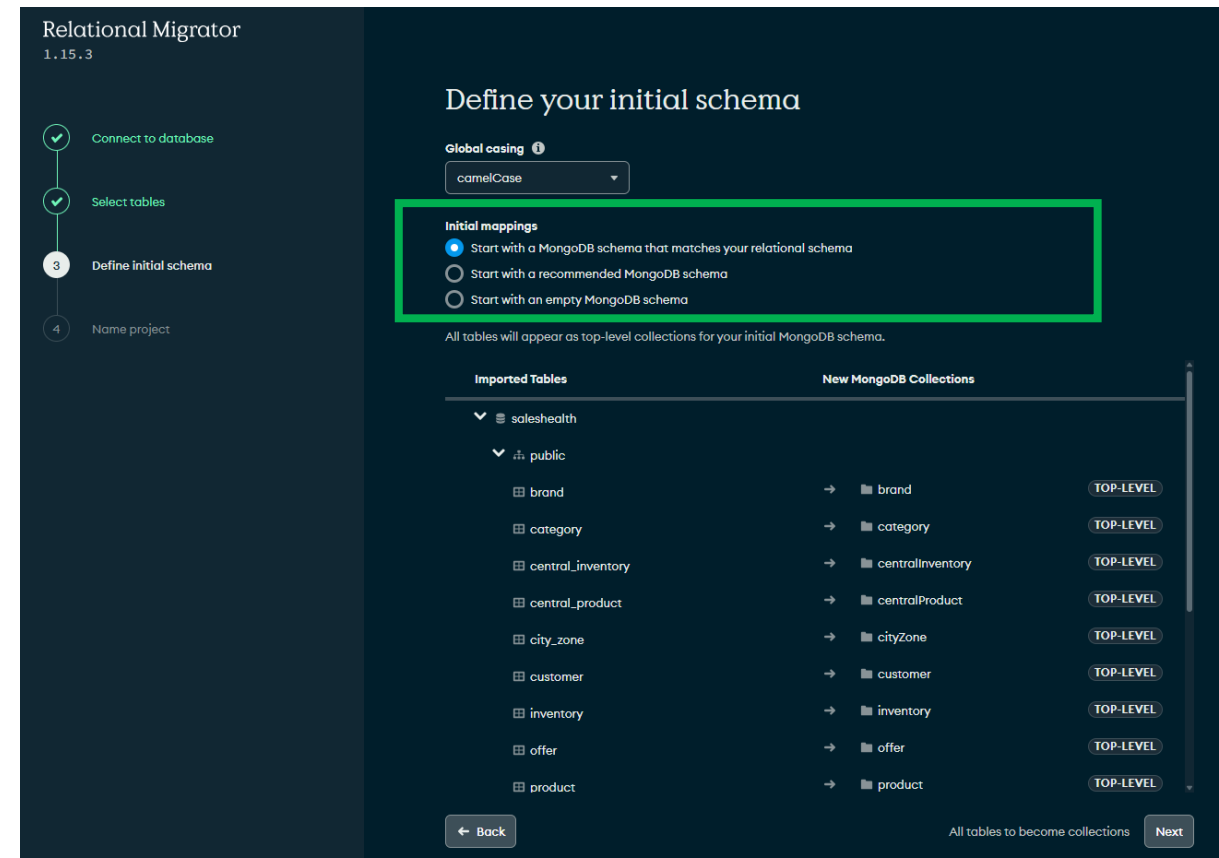
Clear

Test connection

Connect



Seleccionar tablas a migrar



Migrar tal cual o con el esquema recomendado de MongoDB. Los nombres de las colecciones seguirán la notación “camelCase”.

Relational Migrator

1.15.3



Connect to database



Select tables



Define initial schema



Name project

Enter project name

Organize your projects by giving your project a name. Your project file will be stored in the Migrator project directory folder.

Project name

← Back

Done

🏠 saleshealth ⚙️

Pre-Migration Analysis Mapping JSON Schema Data Migration

🔍 Get help

Relational DB: localhost-PostgreSQL MongoDB: Select connection

Schema model

Filter

MANAGE

Relational

- public 17
 - brand
 - category
 - central_inventory
 - central_product
 - city_zone
 - customer
 - inventory
 - offer
 - product

MongoDB

- Database 17
 - brand
 - category
 - centralInventory
 - centralProduct
 - cityZone
 - customer
 - inventory
 - offer
 - product
 - productOffer
 - returnItem

VIEW REL MDB

Main Diagram

bin_code varchar

unit_cost unit_price numeric numeric

warehouse

- warehouse_id serial
- name varchar
- address varchar
- city varchar
- postal_code varchar
- latitude numeric
- longitude numeric

brand

- brand_id serial
- name varchar
- country varchar
- website varchar

category

- category_id serial
- name varchar
- description text

returnItem

- sale_item_id int4

brand

- _id object_id
- brandId integer
- name string
- country string
- website string

category

- _id object_id
- categoryId integer
- name string
- description string

warehouse

- _id object_id
- warehouseId integer
- name string
- address string
- city string
- postalCode integer
- latitude string
- longitude string

Mappings

MongoDB collection

Name

brand

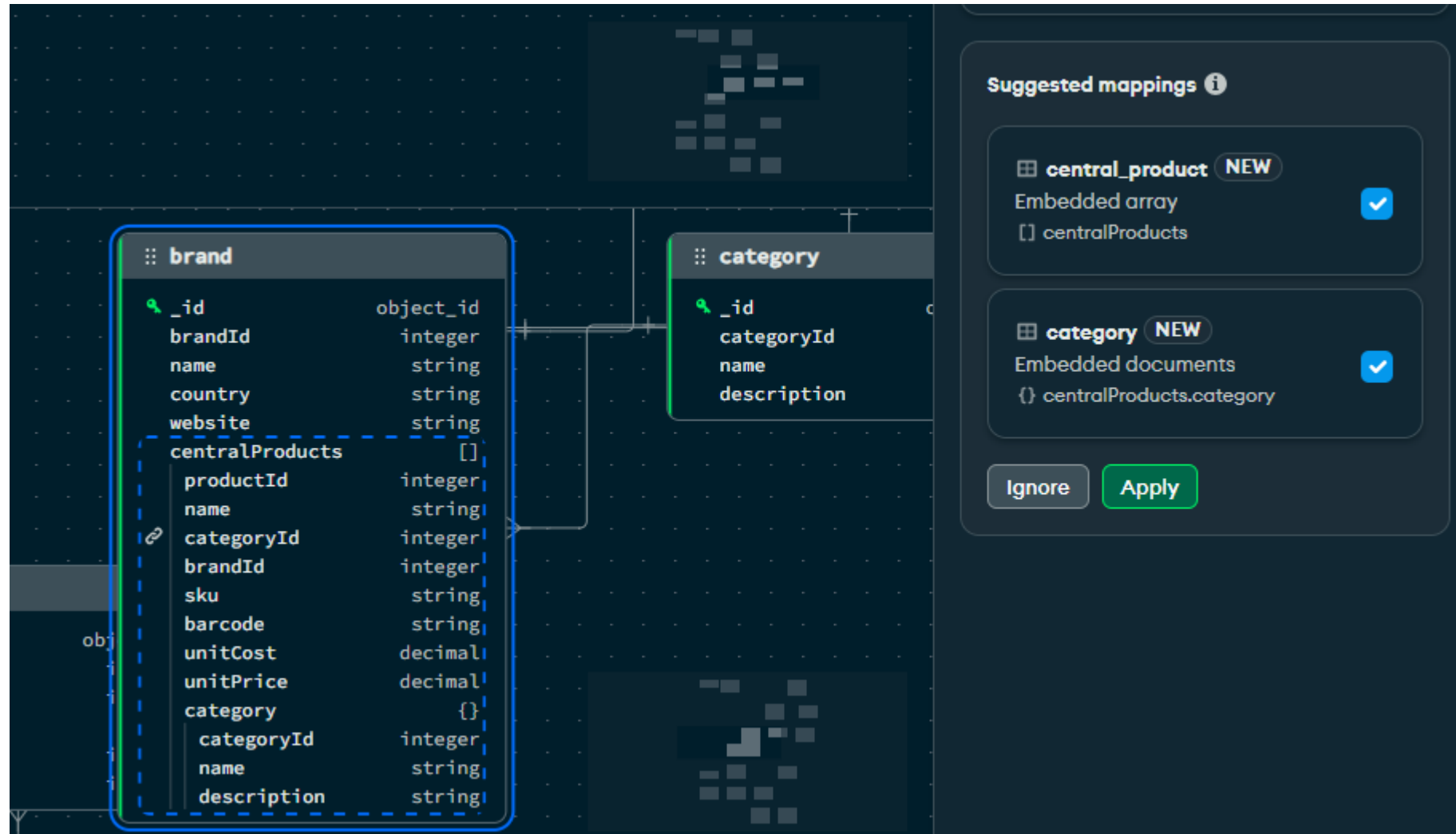
Mappings from relational tables

+ Add

brand

- New documents

Suggested mappings 2

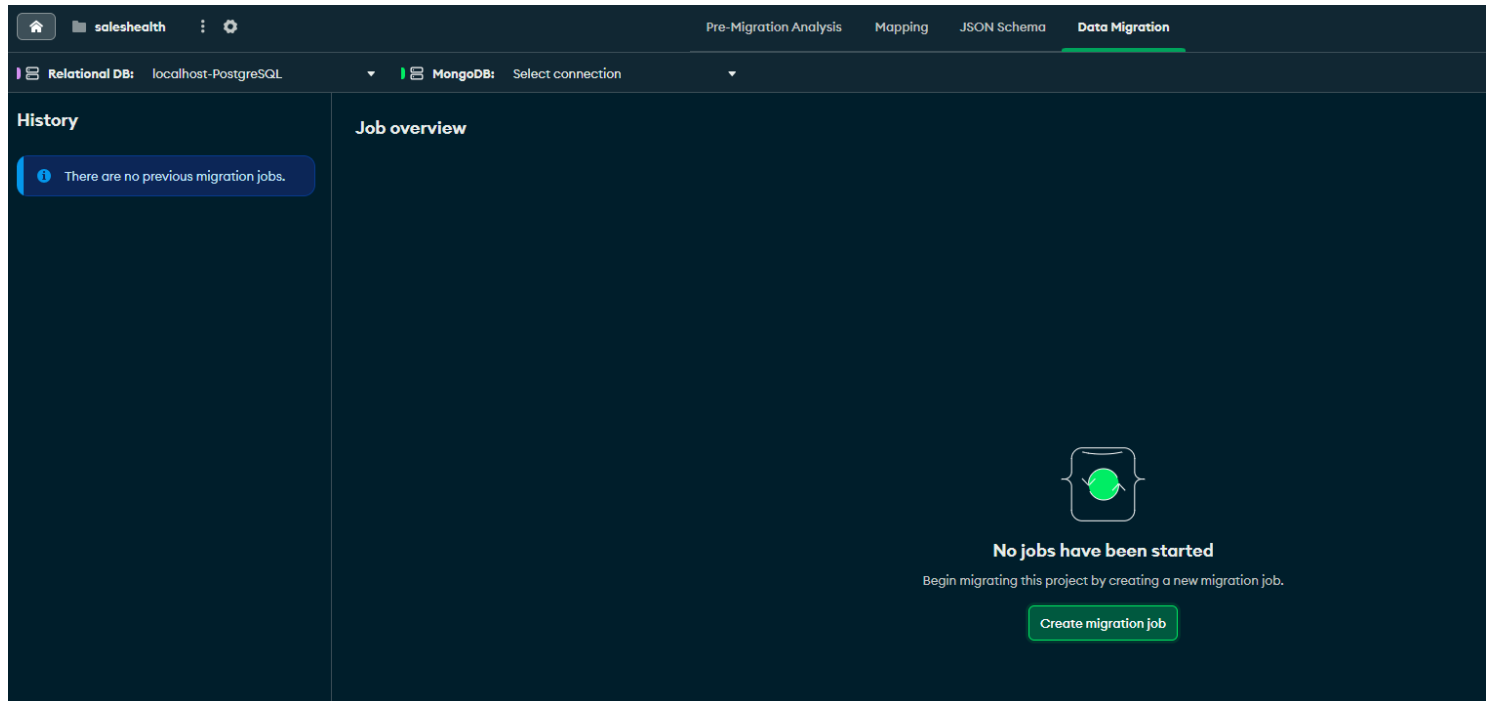


Las sugerencias van a crear datos anidados en estructuras propias de una base de datos de documentos.

The screenshot shows the 'saleshealth' application interface. At the top, there are navigation tabs: 'Pre-Migration Analysis', 'Mapping', 'JSON Schema' (which is highlighted), and 'Data'. Below the navigation bar, there are two database selection options: 'Relational DB: localhost-PostgreSQL' and 'MongoDB: Select connection'. The main content area is divided into two panels. The left panel, titled 'JSON Schema', contains instructions to 'View the JSON schema or example documents for your MongoDB collection.' It has a 'Template' section with two radio buttons: 'Sample documents' (selected) and 'JSON Schema'. Below this is a checkbox for 'All MongoDB collections' which is checked. There is a search bar with a magnifying glass icon and the text 'Filter'. At the bottom of the left panel is a list of collections with checkboxes: 'brand' (checked and highlighted), 'category' (checked), 'centralInventory' (checked), and 'centralProduct' (checked). The right panel displays the JSON schema for the 'brand' collection, titled 'brand_SampleDocument.json'. It shows a single document structure:

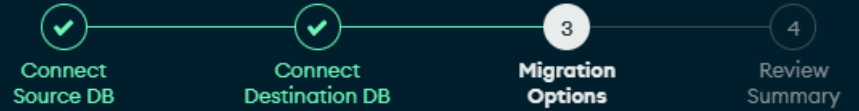
```
{  "_id": ObjectId("63f3823069d05c402a586cfc"),  "brandId": 1000,  "name": "example",  "country": "example",  "website": "example"}
```

Podemos comprobar los esquemas de documentos para cada colección.



Creamos un trabajo de migración, rellenamos los datos de conexión y comenzamos la migración. La verificación de datos es una opción más segura, pero requiere más tiempo.

New Migration Job



Migration Options

Specify your migration options.

Mode

☒ Snapshot ☐ Continuous

! The source database is not configured for running snapshot migration jobs. Consequently, this migration job might fail. You can generate an SQL script providing the required commands needed to configure your source database.

[GENERATE SCRIPT](#)

Rows

☒ Drop destination collections before migration

Errors

☐ Stop migration after errors

Data Verification

☐ Verify migrated data

After the migration is complete, the engine will check that the migrated data in the MongoDB database is equivalent to the data in the source database.

[Learn more](#)

[← Back](#)

[Cancel](#)

[Review Summary](#)

🏠

saleshealth

⋮

⚙️

Pre-Migration Analysis

Mapping

JSON Schema

Data Migration

🔗 Get help

Relational DB:

localhost-PostgreSQL

MongoDB:

clustertestapwi.zp6gl.mongodb....

History

▼ Today

13/4/2026 12:46 19 sec

Job overview

Create migration job

Source URI:

jdbc:postgresql://localhost:5432/saleshealth

Destination URI:

mongodb+srv://CarmenUserTest:*****@clustertestapwi.zp6gl.mongodb.net/saleshealthM

Job Mode:

Snapshot

▼ Snapshot stage

RUNNING

Started:

Today at 12:46:12

Duration:

19 sec

Tables migrated:

12 / 17 (Processing: saleshealth.public.sale)

Rows migrated:

26.818 / 68.666 (39%)

Estimated time remaining:

Calculating...

Snapshot in progress

Terminate

Job update log

Download Log

Date & Time	Severity	Event
> 13/4/2026 12:46:31	Information	Processed snapshot of table: saleshealth.public.return_item
> 13/4/2026 12:46:31	Information	Processed snapshot of table: saleshealth.public.return_reason
> 13/4/2026 12:46:26	Information	Processed snapshot of table: saleshealth.public.offer
> 13/4/2026 12:46:26	Information	Processed snapshot of table: saleshealth.public.product
> 13/4/2026 12:46:26	Information	Processed snapshot of table: saleshealth.public.product_offer
> 13/4/2026 12:46:26	Information	Processed snapshot of table: saleshealth.public...

▼ Snapshot stage

COMPLETED

Started:

Today at 12:46:12

Duration:

39 sec

Tables migrated:

17 / 17

Rows migrated:

71.934

Esperar a que complete el proceso y comprobar en Compass o Atlas.

clustertestapwi.zp6g1.mongodb.net > saleshealthM

> Open MongoDB shell

+ Create collection

↻ Refr

Collection name ⌵	Properties ⌵	Storage size ⌵	Data size ⌵	Documents ⌵	Avg. document size ⌵	Indexes ⌵	Total index size ⌵
brand	-	20.48 kB	2.99 kB	29	103.00 B	2	40.96 kB
category	-	20.48 kB	693.00 B	6	115.00 B	2	40.96 kB
centralInventory	-	20.48 kB	7.30 kB	49	149.00 B	2	40.96 kB
centralProduct	-	20.48 kB	9.22 kB	49	188.00 B	2	40.96 kB
cityZone	-	20.48 kB	6.56 kB	42	156.00 B	2	40.96 kB
customer	-	397.31 kB	1.10 MB	5.8K	191.00 B	2	483.33 kB
inventory	-	40.96 kB	98.00 kB	1K	98.00 B	2	110.59 kB
offer	-	20.48 kB	170.00 B	1	170.00 B	2	40.96 kB
product	-	20.48 kB	8.26 kB	50	165.00 B	2	40.96 kB
productOffer	-	20.48 kB	300.00 B	6	50.00 B	2	40.96 kB
returnItem	-	86.02 kB	233.00 kB	2.3K	100.00 B	2	217.09 kB
returnReason	-	20.48 kB	463.00 B	6	77.00 B	2	40.96 kB
sale	-	798.72 kB	2.08 MB	20K	104.00 B	2	1.62 MB
saleItem	-	2.17 MB	6.00 MB	43K	141.00 B	2	4.67 MB
store	-	4.10 kB	4.11 kB	20	205.00 B	2	8.19 kB
warehouse	-	4.10 kB	195.00 B	1	195.00 B	2	8.19 kB
warehouseLocation	-	4.10 kB	4.59 kB	40	114.00 B	2	8.19 kB

MongoDB – Documentos

Buscar en documentos

`find()`

`find().limit(n)`

`find({'campo':'valor'})`

- Buscar por valor
- Buscar dentro de arrays
- Buscar dentro de objetos (documentos anidados)

`find({'campo':'valor'}, { campo1, campo2 }):`

proyección para determinar que campos incluir

En MongoDB, **la función find()** es el método más común para consultar documentos.

Se utiliza para realizar búsquedas simples, aplicar filtros, seleccionar campos y limitar resultados.

Es rápido, directo y muy útil cuando solo se necesita recuperar documentos tal como están almacenados en la base de datos.

db.collection.find()

Uso	Ejemplo de consulta	Descripción
Búsqueda exacta	{ nombre: "Ana" }	Busca documentos donde el campo 'nombre' sea exactamente 'Ana'.
Comparación (\$gt)	{ edad: { \$gt: 25 } }	Busca documentos donde 'edad' sea mayor que 25.
Comparación múltiple	{ edad: { \$gte: 25, \$lte: 30 } }	Busca documentos con edad entre 25 y 30 inclusive.
Operador lógico \$or	{ \$or: [{ ciudad: "Madrid" }, { edad: { \$lt: 20 } }] }	Coincide si ciudad es 'Madrid' o edad < 20.
Existencia de campo	{ email: { \$exists: true } }	Encuentra documentos que tienen el campo 'email'.
Tipo de dato	{ edad: { \$type: "int" } }	Filtra documentos donde 'edad' es un entero.
Campo anidado	{ "direccion.ciudad": "Madrid" }	Busca documentos con ciudad dentro de direccion = 'Madrid'.
Valor en array	{ intereses: "música" }	Busca documentos que tienen 'música' dentro del array 'intereses'.
Tamaño exacto de array	{ intereses: { \$size: 2 } }	Busca documentos donde el array 'intereses' tiene exactamente 2 elementos.
Valor nulo	{ email: null }	Busca documentos donde el campo 'email' sea null o no exista.
Expresión regular	{ nombre: /^A/ }	Busca nombres que empiezan por 'A'.
Proyección (incluir campos)	{}, { nombre: 1, edad: 1, _id: 0 }	Muestra solo 'nombre' y 'edad' sin el _id.
Ordenar resultados	find().sort({ edad: -1 })	Ordena documentos por 'edad' en orden descendente.
Limitar resultados	find().limit(5)	Devuelve solo los primeros 5 documentos.
Paginar resultados	find().skip(5).limit(5)	Salta los primeros 5 y muestra los siguientes 5.

MongoDB – Documentos

Ejemplo: realizar las siguientes consultas en la consola de Mongo y utilizando Python

1. Encontrar clientes con el nombre María.
2. Encontrar clientes llamados María Martínez.
3. Mostrar del caso anterior nombre, apellidos y email.
4. Ordenar alfabéticamente.
5. Mostrar sólo los 3 primeros resultados.
6. Mostrar los tres siguientes.

MongoDB – Documentos

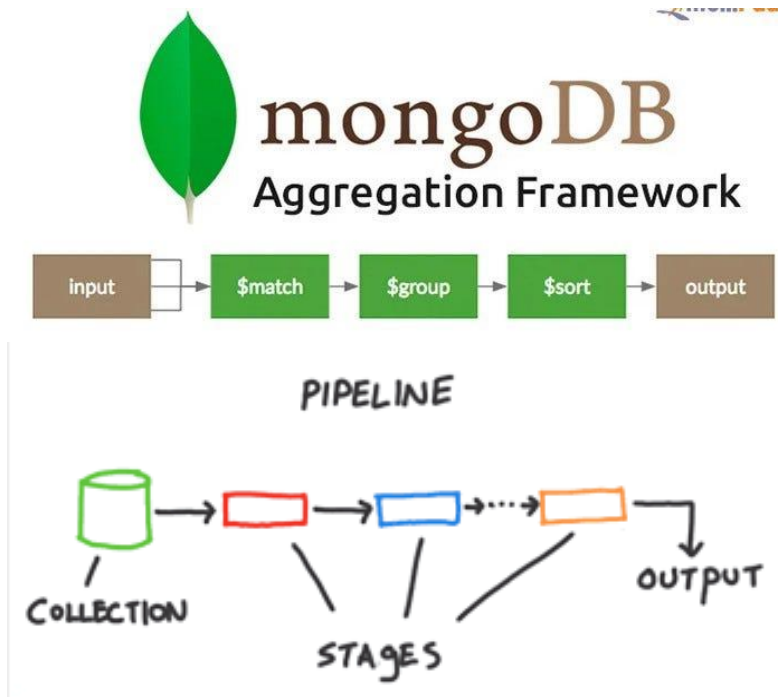
Cuando se requieren operaciones más complejas como agrupar datos, realizar cálculos, transformar estructuras, combinar colecciones o crear nuevos campos, `find()` se queda corto.

Para esos casos, MongoDB ofrece la función **`aggregate()`**, que permite construir **un pipeline de etapas de procesamiento**, similar a cómo se haría en SQL con GROUP BY, JOIN, CASE, funciones de agregación, y más.

El método `aggregate()` trabaja por etapas y permite:

- filtrar (`$match`),
- agrupar (`$group`),
- ordenar (`$sort`),
- transformar campos (`$project`),
- condicionar (`$cond`),
- combinar colecciones (`$lookup`),
- guardar resultados (`$merge`, `$out`).

MongoDB – Documentos



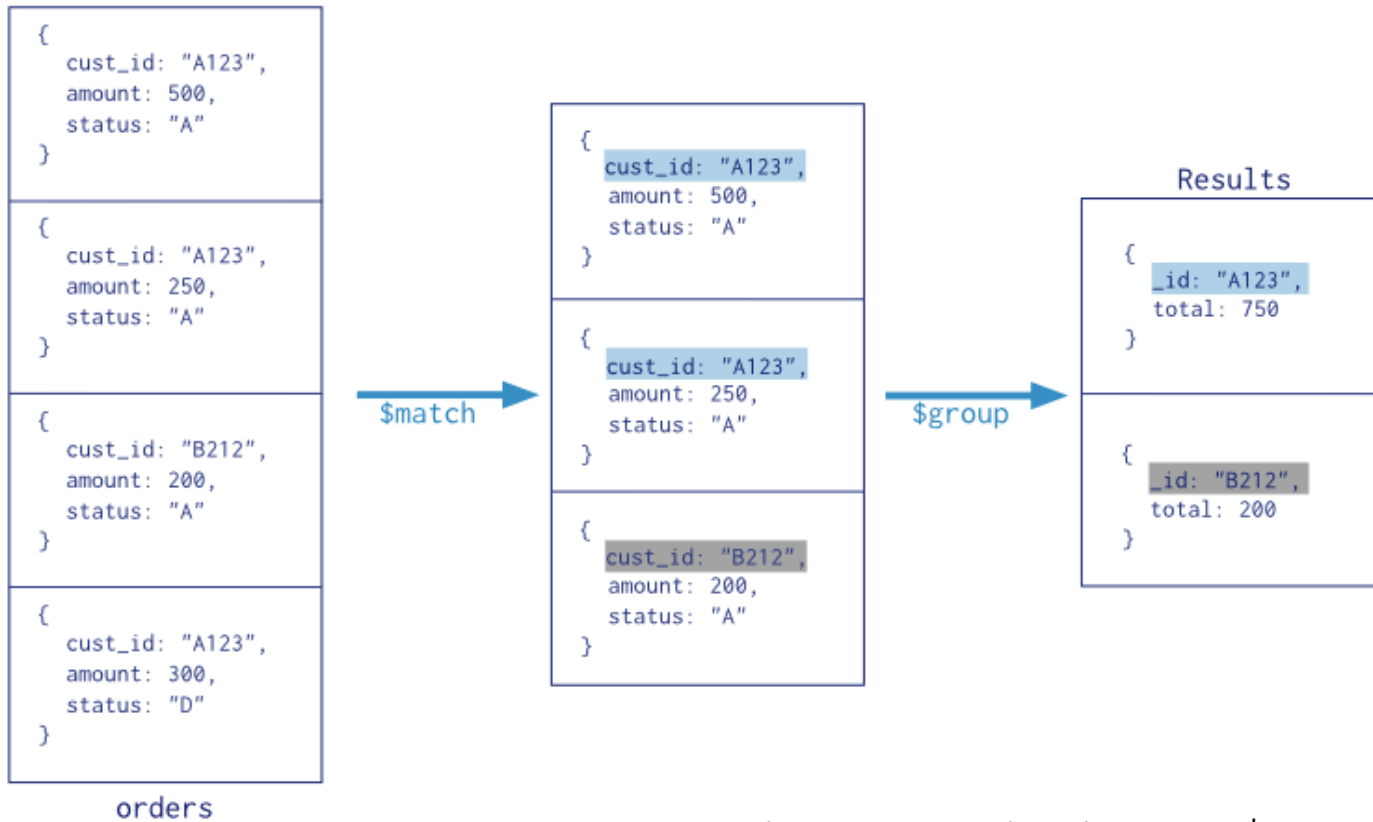
Formato de la consulta de agregación:

```
db.coleccion.aggregate([op1, op2, ... opN])
```

(array de fases)

- Al realizar un pipeline dividimos las consultas en fases, de modo que la salida de una fase es la entrada de la siguiente.
- Cada fase utiliza un operador para realizar una transformación.
- Aunque no hay límite en el número de fases en una consulta, el orden importa en el rendimiento.
- Las fases se pueden repetir, una consulta puede repetir operadores para encadenar diferentes acciones.

Collection
↓
`db.orders.aggregate( [`
 `$match phase → { $match: { status: "A" } },`
 `$group phase → { $group: { _id: "$cust_id", total: { $sum: "$amount" } } }`
 `] )`



Ejemplo de pipeline con \$match y \$group

Operadores de agregación

Operador	Descripción
\$project	Proyección de campos, es decir, propiedades en las que estamos interesados. También nos permite modificar un documento, o crear un subdocumento (reshape). Se utiliza para modificar el conjunto de datos de entrada, agregando, eliminando o recalculando campos para alterar la salida.
\$match	Filtrado de campos, similar a where, para reducir el número de documentos, conservando únicamente aquellos que satisfacen las condiciones especificadas.
\$group	Para agrupar los datos según el campo especificado, similar a group by.
\$sort	Ordenar.
\$skip, \$limit	Omitir un número específico de registros / Limitar los resultados al valor indicado.
\$lookup	Ejecuta un left outer join añadiendo un nuevo campo array a cada documento. Para colecciones en la misma base de datos.
\$unwind	Separa los datos que hay dentro de un array para devolverlo como documentos separados. Por ejemplo, tras lookup.
\$out	Guardar los resultados como una nueva colección.
\$cond	Si cumple la condición (primera expresión) devuelve la segunda expresión, si no devuelve la tercera expresión. Es un operador ternario.

saleshealthM

```
db.sale.aggregate({
  $project: {
    _id : 0,
    saleId : 1,
    total : 1,
    saleKind : {
      $cond : { if : { $gt : ["$total", 500]},
then : "BIG", else : "NORMAL" }
    }
  }
})
```

\$project: determina los campos a devolver y permite usar condicionales con \$cond

```
< {
  total: Decimal128('359.98'),
  saleId: 3,
  saleKind: 'NORMAL'
}
{
  total: Decimal128('399.99'),
  saleId: 4,
  saleKind: 'NORMAL'
}
{
  total: Decimal128('399.84'),
  saleId: 5,
  saleKind: 'NORMAL'
}
{
  total: Decimal128('549.75'),
  saleId: 6,
  saleKind: 'BIG'
}
{
  total: Decimal128('119.90'),
  saleId: 7,
  saleKind: 'NORMAL'
}
{
  total: Decimal128('741.65'),
  saleId: 8,
  saleKind: 'BIG'
}
}
```

Python

```
#Seleccionar / Crear bd y colección
db = client["saleshealthM"]
col_sale = db["sale"]

# Pipeline de agregación equivalente
pipeline = [
    {
        "$project": {
            "_id": 0,
            "saleId": 1,
            "total": 1,
            "saleKind": {
                "$cond": {
                    "if": { "$gt": ["$total", 500] },
                    "then": "BIG",
                    "else": "NORMAL"
                }
            }
        }
    }
]

results = list(col_sale.aggregate(pipeline))

for result in results[:10]:
    print(result)
```

```
{'total': Decimal128('359.98'), 'saleId': 3, 'saleKind': 'NORMAL'}
{'total': Decimal128('399.99'), 'saleId': 4, 'saleKind': 'NORMAL'}
{'total': Decimal128('399.84'), 'saleId': 5, 'saleKind': 'NORMAL'}
{'total': Decimal128('549.75'), 'saleId': 6, 'saleKind': 'BIG'}
{'total': Decimal128('119.90'), 'saleId': 7, 'saleKind': 'NORMAL'}
{'total': Decimal128('741.65'), 'saleId': 8, 'saleKind': 'BIG'}
{'total': Decimal128('1014.93'), 'saleId': 9, 'saleKind': 'BIG'}
{'total': Decimal128('1599.93'), 'saleId': 10, 'saleKind': 'BIG'}
{'total': Decimal128('1209.48'), 'saleId': 11, 'saleKind': 'BIG'}
{'total': Decimal128('39.98'), 'saleId': 12, 'saleKind': 'NORMAL'}
```

PSQL

```
SELECT
  s.sale_id, s.total,
  CASE
    WHEN s.total > 500 THEN 'BIG'
    ELSE 'NORMAL'
  END AS sale_kind
FROM
  sale s;
```

MongoDB

```
db.sale.aggregate({
  $project: {
    _id : 0,
    saleId : 1,
    total : 1,
    saleKind : {
      $cond : { if : { $gt : ["$total", 500]}, then : "BIG",
else : "NORMAL"}
    }
  }
})
```

saleshealthM

```
db.sale.aggregate([
  { $match: { customerId : { $lte : 5 } } },
  { $lookup: {
    from: "customer",
    localField: "customerId",
    foreignField: "customerId",
    as: "cliente"
  } }
])
```

\$match: filtra ventas por id_cliente (<=5)

\$lookup: hace un join con la tabla clientes, cuyos datos devuelve en forma de array

```
{
  _id: ObjectId('69dcc986a87ab871e0e5a4ed'),
  total: Decimal128('324.60'),
  saleId: 394,
  customerId: 2,
  storeId: 8,
  saleDate: 2023-10-11T06:54:35.842Z,
  cliente: [
    {
      _id: ObjectId('69dcc977a87ab871e0e56814'),
      email: 'alejandro.lópez2@example.com',
      phone: '+34 669926134',
      customerId: 2,
      firstName: 'Alejandro',
      lastName: 'López',
      lastName2: 'López',
      createdAt: 2018-09-07T20:45:28.751Z
    }
  ]
}
{
  _id: ObjectId('69dcc986a87ab871e0e5937c'),
  total: Decimal128('269.79'),
  saleId: 5946,
  customerId: 5,
  storeId: 1,
  saleDate: 2024-01-26T03:30:52.376Z
}
```

saleshealthM

```
db.sale.aggregate([
  { $match: { customerId : { $lte : 5 } } },
  { $lookup: {
    from: "customer",
    localField: "customerId",
    foreignField: "customerId",
    as: "cliente"
  } },
  { $unwind: "$cliente" }
])
```

\$unwind convierte array en documentos, aunque sólo haya un resultado en el array.

```
{
  _id: ObjectId('69dcc986a87ab871e0e5a4ed'),
  total: Decimal128('324.60'),
  saleId: 394,
  customerId: 2,
  storeId: 8,
  saleDate: 2023-10-11T06:54:35.842Z,
  cliente: [
    {
      _id: ObjectId('69dcc977a87ab871e0e56814'),
      email: 'alejandro.lópez2@example.com',
      phone: '+34 669926134',
      customerId: 2,
      firstName: 'Alejandro',
      lastName: 'López',
      lastName2: 'López',
      createdAt: 2018-09-07T20:45:28.751Z
    }
  ]
}
{
  _id: ObjectId('69dcc986a87ab871e0e5937c'),
  total: Decimal128('269.79'),
  saleId: 5946,
  customerId: 5,
  storeId: 1,
  saleDate: 2024-01-26T03:30:52.376Z
}
```

```
{
  _id: ObjectId('69dcc986a87ab871e0e5a4ed'),
  total: Decimal128('324.60'),
  saleId: 394,
  customerId: 2,
  storeId: 8,
  saleDate: 2023-10-11T06:54:35.842Z,
  cliente: {
    _id: ObjectId('69dcc977a87ab871e0e56814'),
    email: 'alejandro.lópez2@example.com',
    phone: '+34 669926134',
    customerId: 2,
    firstName: 'Alejandro',
    lastName: 'López',
    lastName2: 'López',
    createdAt: 2018-09-07T20:45:28.751Z
  }
}
{
  _id: ObjectId('69dcc986a87ab871e0e5937c'),
  total: Decimal128('269.79'),
  saleId: 5946,
  customerId: 5,
  storeId: 1,
}
```



```
db.sale.aggregate([
  { $match: { customerId : { $lte : 5 } } },
  { $lookup: {
    from: "customer",
    localField: "customerId",
    foreignField: "customerId",
    as: "cliente"
  } },
  { $unwind: "$cliente" },
  { $group : {
    _id: "$cliente.customerId",
    name : { $first: "$cliente.firstName" },
    surname : { $first: "$cliente.lastName" },
    total: { $sum : "$total" },
    nsales : { $sum : 1 },
    avgsales : { $avg: "$total" }
  } }
])
```

\$group permite agrupar y hacer cálculos en los agrupados.

```
{
  _id: 3,
  name: 'Alejandro',
  surname: 'López',
  total: Decimal128('19862.51'),
  nsales: 24,
  avgsales: Decimal128('827.604583333333333333333333333333')
}

{
  _id: 4,
  name: 'Javier',
  surname: 'Ruiz',
  total: Decimal128('8203.76'),
  nsales: 10,
  avgsales: Decimal128('820.376')
}

{
  _id: 5,
  name: 'David',
  surname: 'Gómez',
  total: Decimal128('10938.05'),
  nsales: 22,
  avgsales: Decimal128('497.1840909090909090909090909090909')
}

{
  _id: 2,
```

Total, número de ventas y valor promedio de venta por clientes para clientes con id <=5

PSQL

```
SELECT c.customer_id,  
       c.first_name as name,  
       c.last_name as surname,  
       SUM(s.total) as total,  
       COUNT(*) as nsales,  
       AVG(s.total) as avgsales  
FROM sale s  
LEFT JOIN customer c  
  on c.customer_id = s.customer_id  
WHERE s.customer_id <= 5  
GROUP BY c.customer_id
```

MongoDB

```
db.sale.aggregate([  
  { $match: { customerId : {$lte : 5} } },  
  { $lookup: {  
    from: "customer",  
    localField: "customerId",  
    foreignField: "customerId",  
    as: "cliente"  
  } },  
  { $unwind: "$cliente" },  
  { $group : {  
    _id: "$cliente.customerId",  
    name : { $first: "$cliente.firstName" },  
    surname : { $first: "$cliente.lastName" },  
    total: { $sum : "$total" },  
    nsales : { $sum : 1 },  
    avgsales : { $avg: "$total" }  
  } }  
])
```

Facet (&bucket): agregación de facetas

Permite caracterizar los datos en varias facetas o dimensiones en paralelo.

Para realizar múltiples agregaciones o análisis en una única consulta (> eficiencia).

```
{ $facet:
  {
    <outputField1>: [ <stage1>, <stage2>, ... ],
    <outputField2>: [ <stage1>, <stage2>, ... ],
    ...
  }
}
```



Se ejecutan sub-pipelines:

- Son totalmente independientes entre sí.
- Su ejecución es secuencial en un único hilo de programación.

Facet (&bucket): agregación de facetas



Se ejecutan sub-pipelines:

- Son totalmente independientes entre si.
- Su ejecución es secuencial en un único hilo de programación.

Tener en cuenta:

- * Los datos de entrada son los mismos para todas las sub-pipelines.
- * Los datos de salida de una sub-pipeline no pueden usarse como datos de entrada de la siguiente.

\$bucket: permite categorizar los datos entrantes en grupos (llamados buckets), con un límite de 100 megabytes de RAM (si necesita más memoria falla).

saleshealth > product

```
_id: ObjectId('69dce093bb9cd9f6fad8ab54')
product_id: 1
name: "Tensiómetro digital"
▼ category: Object
  category_id: 1
  category: "Diagnóstico"
▼ manufacturer: Object
  brand_id: 1
  brand: "Omron"
  country: "Japón"
price: 59.99
created_at: 2026-04-04T11:23:48.591+00:00
```

```
db.product.aggregate([
  { $facet : {
    "byPrice": [
      { $bucketAuto : {
        groupBy: "$price",
        buckets: 3
      } }
    ],
    "byCategory": [
      { $group : {
        _id: "$category.category_id",
        category : { $first: "$category.category" },
        nitems : { $sum:1 }
      } }
    ],
    "byBrand": [
      { $group : {
        _id: "$manufacturer.brand_id",
        brand : { $first: "$manufacturer.brand" },
        nitems : { $sum:1 }
      } }
    ],
    "byBrandCountry": [
      { $group : {
        _id: "$manufacturer.country",
        nitems : { $sum:1 }
      } }
    ]
  } }
])
```

```
db.product.aggregate([
  { $facet : {
    "byPrice": [
      { $bucketAuto : {
        groupBy : "$price",
        buckets: 3
      } }
    ],
    "byCategory" : [
      { $group : {
        _id: "$category.category_id",
        category : { $first: "$category.category" },
        nitems : { $sum: 1 }
      } }
    ],
    "byBrand" : [
      { $group : {
        _id: "$manufacturer.brand_id",
        brand : { $first: "$manufacturer.brand" },
        nitems : { $sum: 1 }
      } }
    ],
    "byBrandCountry" : [
      { $group : {
        _id: "$manufacturer.country",
        nitems : { $sum: 1 }
      } }
    ]
  } }
])
```

```
{
  byPrice: [
    {
      _id: {
        min: 19.99,
        max: 69.9
      },
      count: 17
    },
    {
      _id: {
        min: 69.9,
        max: 139.99
      },
      count: 17
    },
    {
      _id: {
        min: 139.99,
        max: 399.99
      },
      count: 16
    }
  ],
```

```
byCategory: [
  {
    _id: 5,
    category: 'Higiene',
    nitems: 4
  },
  {
    _id: 2,
    category: 'Wellness',
    nitems: 14
  },
  {
    _id: 3,
    category: 'Movilidad',
    nitems: 5
  },
  {
    _id: 6,
    category: 'Tratamiento',
    nitems: 5
  },
  {
    _id: 1,
    category: 'Diagnóstico',
    nitems: 14
  },
  {
    _id: 4,
    category: 'Rehabilitación',
    nitems: 7
  },
  {
    _id: null,
    category: null,
    nitems: 1
  }
]
```

```
byBrand: [
  {
    _id: 4,
    brand: 'Xiaomi',
    nitems: 4
  },
  {
    _id: 13,
    brand: 'Thermoval',
    nitems: 2
  },
  {
    _id: 23,
    brand: 'Seca',
    nitems: 1
  },
  {
    _id: 19,
    brand: 'Dyson',
    nitems: 1
  },
  {
    _id: 5,
    brand: 'Abbott',
    nitems: 2
  },
  {
    _id: 8,
    brand: 'Naipo',
    nitems: 2
  },
  {
    _id: 7,
    brand: 'Philips',
    nitems: 2
  }
]
```

```
byBrandCountry: [
  {
    _id: 'Alemania',
    nitems: 11
  },
  {
    _id: 'Estados Unidos',
    nitems: 12
  },
  {
    _id: 'Suiza',
    nitems: 3
  },
  {
    _id: 'China',
    nitems: 8
  },
  {
    _id: 'República Checa',
    nitems: 1
  },
  {
    _id: 'España',
    nitems: 4
  },
  {
    _id: 'Francia',
    nitems: 2
  },
  {
    _id: 'Japón',
    nitems: 5
  },
  {
    _id: 'Países Bajos',
    nitems: 2
  }
]
```

MongoDB – Archivos

En **MongoDB**, puedes almacenar **fotos, audio, vídeo o cualquier tipo de fichero binario** utilizando una característica llamada **GridFS**. GridFS es el sistema de MongoDB para almacenar archivos grandes o binarios (fotos, vídeos, PDFs, MP3, etc.). Divide los archivos en fragmentos (chunks) y los guarda en dos colecciones:

- fs.files: contiene los metadatos del archivo.
- fs.chunks: contiene los datos binarios divididos en partes.

```
import gridfs

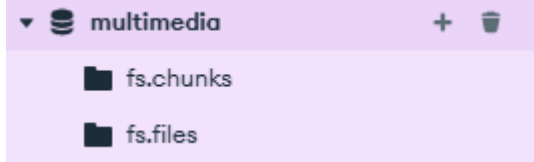
#Seleccionar / Crear bd y colección
db = client["multimedia"]

fs = gridfs.GridFS(db)

# Subir un archivo
with open("data/minions.jpg", "rb") as f:
    file_id = fs.put(f, filename="minions.jpg", type="image")

print("Archivo subido con ID: ", file_id)
```

Por defecto, GridFS utiliza un tamaño de fragmento de 255 KiB; es decir, GridFS divide un archivo en fragmentos de 255 KiB, con la excepción del último fragmento. El último fragmento es tan grande como sea necesario. De manera similar, los archivos que no son más grandes que el tamaño del fragmento solo tienen un fragmento final, y utilizan solo el espacio necesario más algunos metadatos adicionales.



clustertestapwi.zp6g1.mongodb.net > multimedia > fs.chunks

Documents 1 Aggregations Schema Indexes 2 Validation

Type a query: { field: 'value' } or [Generate query](#)

[ADD DATA](#) [UPDATE](#) [DELETE](#) [EXPORT DATA](#) [EXPORT CODE](#)

```
{
  "_id": ObjectId('69dd36e012fbc21fd95402cb'),
  "files_id": ObjectId('69dd36e012fbc21fd95402ca'),
  "n": 0,
  "data": Binary.createFromBase64('/9j/4QC8RXhpZgAASUkqAAgAAAAGABIBAwABAAAAQAAABoBBQ
```

clustertestapwi.zp6g1.mongodb.net > multimedia > fs.files

Documents 1 Aggregations Schema Indexes 2 Validation

Type a query: { field: 'value' } or [Generate query](#)

[ADD DATA](#) [UPDATE](#) [DELETE](#) [EXPORT DATA](#) [EXPORT CODE](#)

```
{
  "_id": ObjectId('69dd36e012fbc21fd95402ca'),
  "filename": "minions.jpg",
  "type": "image",
  "chunkSize": 261120,
  "length": 123299,
  "uploadDate": 2026-04-13T18:33:04.443+00:00
```



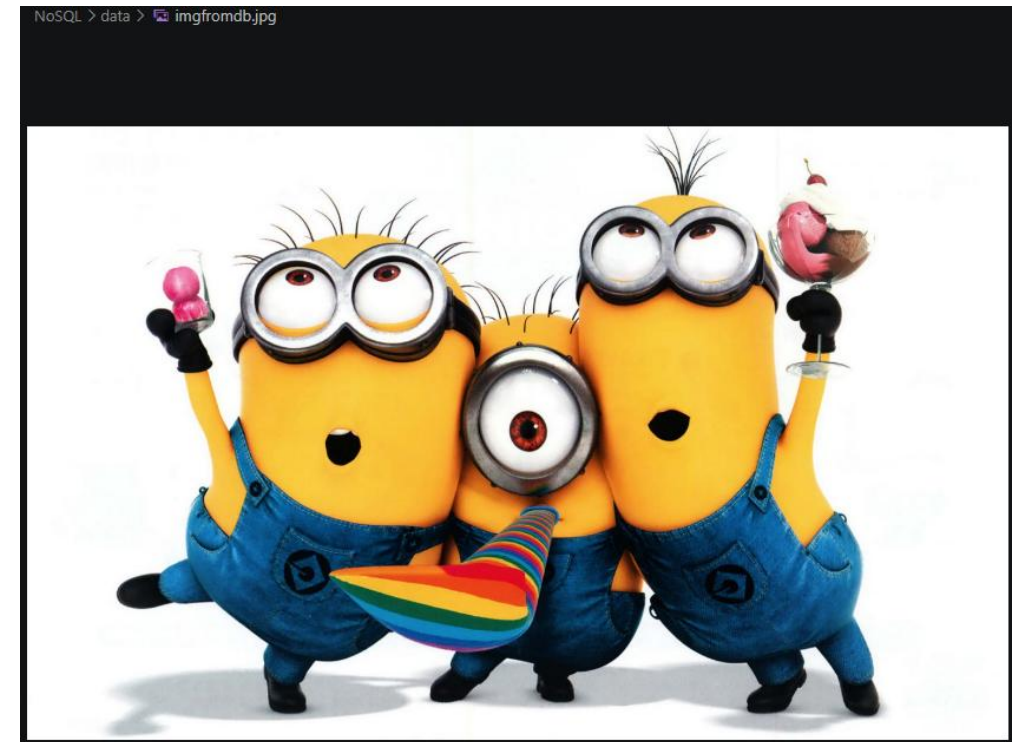
```
"""Leer / descargar archivo desde BD"""

# Buscar por filename o por file_id (ObjectId)
fileindb = fs.get(file_id)

# Leer contenido
filecontent = fileindb.read()

# Guardar en disco local de nuevo
with open("data/imgfromdb.jpg", "wb") as f_out:
    f_out.write(filecontent)
    print("Archivo descargado exitosamente.")
```

- Cuando desee acceder a información de partes de archivos grandes sin tener que cargar archivos completos en la memoria, puede usar GridFS para recuperar secciones de archivos sin leer el archivo completo en la memoria.
- Cuando desee mantener sus archivos y metadatos automáticamente sincronizados e implementados en varios sistemas e instalaciones.



Comentarios y preguntas